

Rechnergestützte Numerik und Simulation

(am Beispiel von Matlab/Simulink)

Implementierung und Matlab-Praxis 3:

Datentypen

Inhalt

3. Datentypen

- 3.1 Numerische Datentypen
- 3.2 Strings
- 3.3 Cell-Arrays und Structs
- 3.4 Besonderheiten
- 3.5 Daten-Import/Export

3. Datentypen – 3.1 Numerische Datentypen

Numerisch :

- Numerisch Variablen sind als Standard von Typ ‚Double‘ und damit intern ein 64-Bit-Fließkomawert, unabhängig von der Darstellung

```
>> x = 323.345;
```

```
>> x = 123.4e9;
```

Darstellungsformate:

```
>> a = sqrt(2);
```

```
>> a
```

```
a =
```

```
1.4142
```

```
>> format long
```

```
>> a
```

```
a =
```

```
1.414213562373095
```



3. Datentypen – 3.1 Numerische Datentypen

Weitere Ausgabeformate

- short skalierte Festpunktdarstellung mit 5 Stellen
 → 0.3679
- long skalierte Festpunktdarstellung mit 15 Stellen
 → 0.36787944117144
- short e Gleitpunktdarstellung mit 5 Stellen
 → 3.6788e-01
- long e Gleitpunktdarstellung mit 15 Stellen
 → 3.678794411714423e-01
- short g optimale Darstellung mit 5 Stellen
 → 0.36788
- long g optimale Darstellung mit 15 Stellen
 → 0.367879441171442
- short eng mindestens 5 Stellen und einen durch drei teilbaren Exponenten
 → 367.8794e-003
- long eng 16 signifikante Stellen und einen durch drei teilbaren Exponenten
 → 367.879441171442e-003
- rat Näherung in Bruchdarstellung
 → 536/1457

3. Datentypen – 3.1 Numerische Datentypen

- Durch das Schlüsselwort `single` ist es möglich, den Typ ‚Single‘ als 32-Bit-Fließkomawert zu erzwingen:

```
>> format long  
>> a = single(sqrt(2))
```

```
a =
```

```
1.4142135
```

```
>> a = (sqrt(2))
```

```
a =
```

```
1.414213562373095
```



3. Datentypen – 3.1 Numerische Datentypen

```
>> realmax('single')
```

→ Liefert den größten darstellbaren Wert

```
ans =
```

```
3.4028235e+038
```

```
>> realmax('double')
```

```
ans =
```

```
1.797693134862316e+308
```

Siehe auch `realmin('double')` .

3. Datentypen – 3.1 Numerische Datentypen

Auf die gleiche Weise können auch die Integer Variablen definiert werden:

<code>>>a = int8(12);</code>	→ 8-Bit signed Integer
<code>>>a = int16(12);</code>	→ 16-Bit signed Integer
<code>>>a = int32(12);</code>	→ 32-Bit signed Integer
<code>>>a = int64(12);</code>	→ 64-Bit signed Integer
<code>>>a = uint8(12);</code>	→ 8-Bit unsigned Integer
<code>>>a = uint16(12);</code>	→ 16-Bit unsigned Integer
<code>>>a = uint32(12);</code>	→ 32-Bit unsigned Integer
<code>>>a = uint64(12);</code>	→ 64-Bit unsigned Integer



3. Datentypen – 3.1 Numerische Datentypen

Operationen auf numerische Datentypen:

- Auf Fließkommatypen sind alle arithmetische Operationen definiert.
- Auf Integer-Datentypen sind die meisten arithmetischen Operationen definiert (mit Ausnahme `int64` und `uint64`). Die Operanden müssen den gleichen Integer-Datentyp haben.
- Die meisten arithmetischen Operationen sind auch zwischen Fließkommatypen und Integer-Datentypen definiert, wobei das Ergebnis vom Integer-Datentypen ist.

```
>> int8(4)*2.5
```

```
ans =
```

```
10
```

```
>> int8(4)*2.55
```

```
ans =
```

```
10
```



3. Datentypen – 3.1 Numerische Datentypen

- Boolsche Variablen werden durch das Schlüsselwort `boolean` definiert:

```
>> a = boolean(0)
```

```
a =
```

```
0
```

```
>> a = boolean(4)
```

```
a =
```

```
1
```

```
>> a = true
```

```
a =
```

```
1
```

```
>> a = false
```

```
a =
```

```
0
```



3. Datentypen – 3.1 Numerische Datentypen

- Operationen auf boolesche Variablen:

```
>> true & true
```

```
ans =
```

```
1
```

```
>> true * true
```

```
ans =
```

```
1
```

```
>> true + true
```

```
ans =
```

```
2
```

```
>> true * 2.1
```

```
ans =
```

```
2.1000
```

→ Die booleschen Werte ,true' und ,false' werden numerisch wie 0 und 1 interpretiert



3. Datentypen – 3.1 Numerische Datentypen

- Umgekehrt Zahlwerte ungleich null als 'true' ansonsten als 'false' interpretiert und können auf Logikoperationen angewendet werden

```
>> 1 & 2
```

```
ans =
```

```
1
```

```
>> 1 | 0
```

```
ans =
```

```
1
```

```
>> 3.6 | 0
```

```
ans =
```

```
1
```



3. Datentypen – 3.1 Numerische Datentypen

- Matlab bietet auch den Datentyp ‚Complex‘:

```
>> a=complex(1,2)
```

```
a =
```

```
1.0000 + 2.0000i
```

```
>> a = 1 + i * 2
```

```
a =
```

```
1.0000 + 2.0000i
```

```
>> a = 1 + j * 2
```

```
a =
```

```
1.0000 + 2.0000i
```



3. Datentypen – 3.1 Numerische Datentypen

Operationen auf den Datentyp ‚Complex‘:

```
>> a = 1 + j * 2;  
>> b = 3 + j * 4;  
>> a*b
```

```
ans =
```

```
-5.0000 +10.0000i
```

```
>> imag(a)
```

```
ans =
```

```
2
```

```
>> conj(a)
```

```
ans =
```

```
1.0000 - 2.0000i
```

Weiter Operationen: `real`, `abs`, `angle`, `isreal`



3. Datentypen – 3.1 Numerische Datentypen

- Es sind auch n-dimensionale Arrays möglich:

```
>> A(2,2,2,2) =1
```

```
A(:, :, 1, 1) =
```

```
0    0  
0    0
```

```
A(:, :, 2, 1) =
```

```
0    0  
0    0
```

```
A(:, :, 1, 2) =
```

```
0    0  
0    0
```

```
A(:, :, 2, 2) =
```

```
0    0  
0    1
```

3. Datentypen – 3.1 Numerische Datentypen

- Arrays können auf alle numerischen Datentypen einschließlich boolescher Werte gebildet werden. Beispiel:

```
>> A = 5 > magic(3)
```

```
A =
```

```
0     1     0
1     0     0
1     0     1
```

```
>> A = logical(magic(2))
```

```
A =
```

```
1     1
1     1
```



3. Datentypen – 3.1 Numerische Datentypen

Konstruktor-Methoden:

Im Sinne einer strukturierten Programmierung sollten Arrays egal welchen Datentyps vorab definiert werden.

Beispiele für Konstruktorbefehle:

```
>> ones (2,3)
```

```
ans =
```

```
1    1    1
1    1    1
```

```
>> eye(3)
```

```
ans =
```

```
1    0    0
0    1    0
0    0    1
```



3. Datentypen – 3.1 Numerische Datentypen

```
>> rand(2)
```

```
ans =
```

```
0.8147    0.1270  
0.9058    0.9134
```

```
>> linspace(0,20,11)
```

```
ans =
```

```
0      2      4      6      8     10     12     14     16     18  
20
```

```
>> (1:3:9)
```

```
ans =
```

```
1      4      7
```



3. Datentypen – 3.1 Numerische Datentypen

Spezielle Funktionen für die Erstellung bzw. Manipulation mehrdimensionaler Felder :

Erzeugung/Manipulation mehrdimensionaler Felder

- `rand, randn` Felder mit Pseudo-Zufallszahlen
- `ones, zeros` Felder mit den Elementen 1 bzw. 0
- `reshape, permute` Änderung der Gestalt bzw. Dimensionsreihenfolge
- `squeeze` Entfernt Dimensionen der Länge 1

Funktionen für mehrdimensionale Felder

- `size, length, ndims` Größen-, Längen- und Dimensionsangaben
- `sin, cos,` ... Elementarfunktionen operieren elementweise
- `sum, prod, diff,` Summen-, Produkt-, Differenzenbildung
über die angegebene Dimension



3. Datentypen – 3.2 Strings

Strings:

Strings sind Arrays von Buchstaben:

```
>> gruss = 'Hello World';
```

entspricht

```
>> gruss = ['Hello World'];
```

entspricht

```
>> gruss = ['H','e','l','l','o',' ','W','o','r','l','d']
```

```
gruss =
```

```
Hello World
```



3. Datentypen – 3.2 Strings

Also funktionieren auch die Verknüpfungs- und Indizierungsmethoden:

```
>> gruss(1:5)
```

```
ans =
```

```
Hello
```

```
>> gruss = [gruss, ' ', gruss]
```

```
gruss =
```

```
Hello World Hello World
```

```
>> gruss = [gruss; gruss] (gleichwertig: strvcat(...) )
```

```
gruss =
```

```
Hello World Hello World
```

```
Hello World Hello World
```



3. Datentypen – 3.2 Strings

Wichtige Befehle für Strings:

```
>> Eingabe = 'Eingangswert 1123';  
>> b = 2 * str2num(Eingabe(14:17))
```

b =

2246

```
>> Ergebnis=[ 'Ergebnis: ', num2str(b) ]
```

Ergebnis =

Ergebnis: 2246

```
>> char(65:90)
```

ans =

ABCDEFGHIJKLMNOPQRSTUVWXYZ



3. Datentypen – 3.2 Strings

Ausgabe:

```
>> a = sqrt(2);
```

```
>> disp(a);
```

1.4142

→ Ausgabe ohne Angabe der Variable



3. Datentypen – 3.2 Strings

Andere Ausgabe:

```
>> sprintf('%0.5g', (1+sqrt(5))/2)
    1.618

>> sprintf('%0.5g', 1/eps)
    4.5036e+15

>> sprintf('%15.5f', 1/eps)
    4503599627370496.00000

>> sprintf('%d', round(pi))
    3

>> sprintf('%s', 'hello')
    hello

>> sprintf('The array is %dx%d.', 2, 3)
    The array is 2x3.

>> sprintf('\n') is the line termination character on all
    platforms.
```

3. Datentypen – 3.3 CellArrays und Structs

CellArray:

CellArrays bieten einen Datencontainer für Elemente unterschiedlicher Datentypen. Die Elemente des CellArrys selbst sind dabei als Zeiger auf die eigentlichen Daten zu verstehen. Elementar für die Definition und Indizierung eines CellArrays ist die Schreibweise mit geschweiften Klammern ({}).

```
>> c = {1 , 'Hello', 34.546, [1 2 3]};
```

```
>> c{2}
```

```
ans =
```

```
Hello
```

```
>> c{4}(3)
```

```
ans =
```

```
3
```

3. Datentypen – 3.3 CellArrays und Structs

```
>> c{3} * 2
```

```
ans =
```

```
69.0920
```

```
>> c(3)
```

```
ans =
```

```
[34.5460]
```

```
>> c(3) * 2
```

```
??? Undefined function or method 'mtimes' for input arguments  
of type 'cell'.
```

```
>> cell2mat(c(3)) * 2
```

```
ans =
```

```
69.0920
```

3. Datentypen – 3.3 CellArrays und Structs

- Wichtige Befehle für CellArrays:

```
>> a = cell(2,3)
```

```
a
```

```
    []    []    []  
    []    []    []
```

```
>> a{2,3} = 3
```

```
a =
```

```
    []    []    []  
    []    []    [3]
```

```
>> a(1,2) = 3
```

```
??? Conversion to cell from double is not possible.
```

```
>> a(1,2) = num2cell(3)
```

```
a =
```

```
    []    [3]    []  
    []    []    [3]
```

3. Datentypen – 3.3 CellArrays und Structs

Eckige Klammern definieren eine leere Zelle:

```
>> a{1,4} = []
```

a =

[]	[3]	[]	[]
[]	[]	[3]	[]

3. Datentypen – 3.3 CellArrays und Structs

- **Struktur-Arrays (Structs):**

Struktur-Arrays fassen wie Cell-Arrays unterschiedliche Datentypen in einem Container zusammen. Im Gegensatz zu Cell-Arrays haben Struktur-Array aber eine Strukturierung und Syntax.

- Die Struktur eines Struktur-Arrays ist baumartig.
- Die Trennung der Ebenen erfolgt durch die Punktschreibweise:

```
>> Tiere.Tiger.Raubtier = true;  
>> Tiere.Tiger.gefaehrlich = 'hoch';  
>> Tiere.Tiger.Anzahl = 1000;  
>> Tiere.Kaninchen.Raubtier = 'false';  
>> Tiere.Kaninchen.gefaehrlich = 'niedrig';  
>> Tiere.Kaninchen.Anzahl = inf;  
>> Tiere.Ratte = 'ekelig';  
>> Tiere
```

```
Tiere =  
      Tiger: [1x1 struct]  
      Kaninchen: [1x1 struct]  
      Ratte: 'ekelig'
```



3. Datentypen – 3.3 CellArrays und Structs

```
>> Tiere.Ratte
```

```
ans =
```

```
ekelig
```

```
>> Tiere.Tiger.Anzahl
```

```
ans =
```

```
1000
```

Merke:

Die Tiefe einzelner ‚Äste‘ eines Struktur-Arrays und die Struktur selbst muss nicht konsistent sein.



3. Datentypen – 3.3 CellArrays und Structs

Struktur-Arrays können in der Wurzel oder auf jeder beliebigen Ebene wiederum n-dimensionale Arrays sein:

```
>> personal(1).name = 'Mustermann';  
>> personal(1).gehalt = 4000;  
>> personal(2).name = 'Fliegender Holländer';  
>> personal(2).gehalt = 5000;  
>> personal
```

```
personal =
```

```
1x2 struct array with fields:  
    name  
    gehalt
```

```
>> personal(1)
```

```
ans =
```

```
    name: 'Mustermann'  
    gehalt: 4000
```

```
>> personal(1).name
```

```
ans =
```

```
Mustermann
```



3. Datentypen – 3.3 CellArrays und Structs

```
>> neuerChef = personal(1);  
>> neuerChef.name
```

```
ans =
```

```
Mustermann
```

Es geht auch:

```
>> personal(1).gehaltsmonat(12).ausgezahlt = 2500;
```



3. Datentypen – 3.3 CellArrays und Structs

Erzeugung/Manipulation von Strukturen:

- `struct`
- `setfield, getfield`
- `isfield, isstruct`
- `rmfield`
- `orderfields`

Erstellen von Strukturvariablen

Setzen und Auslesen von Feldern

Indikatorfunktionen für Felder/Strukturen

Löschen eines Feldes

Sortierung der Felder

3. Datentypen – 3.3 CellArrays und Structs

- Typische Anwendung von Struktur-Arrays:
Initialisierung von Variablen für ein Simulink-Modell:

```
>> MDL.Getriebe.Uebersetzung.v = 0.75;  
>> MDL.Getriebe.Uebersetzung.Unit = '-';  
>> MDL.Getriebe.Uebersetzung.Desc = 'Getriebeübersetzung';  
>> MDL.Getriebe.Reibung.v = 0.0012;  
>> MDL.Getriebe.Reibung.Unit = 'Nm';  
>> MDL.Getriebe.Reibung.Desc = 'Reibungsmoment';  
>> MDL.Motor.Leistung.v = 3;  
>> MDL.Motor.Leistung.Unit = 'kW';  
>> MDL.Motor.Leistung.Desc = 'Motorleistung';
```



3. Datentypen – 3.3 CellArrays und Structs

- Das aufrufen von Struktur-Arrays in der Kommandozeile kann man sich durch Benutzung der Tabulatortaste vereinfachen:

>> Tiere. → dann Tabulator-Taste



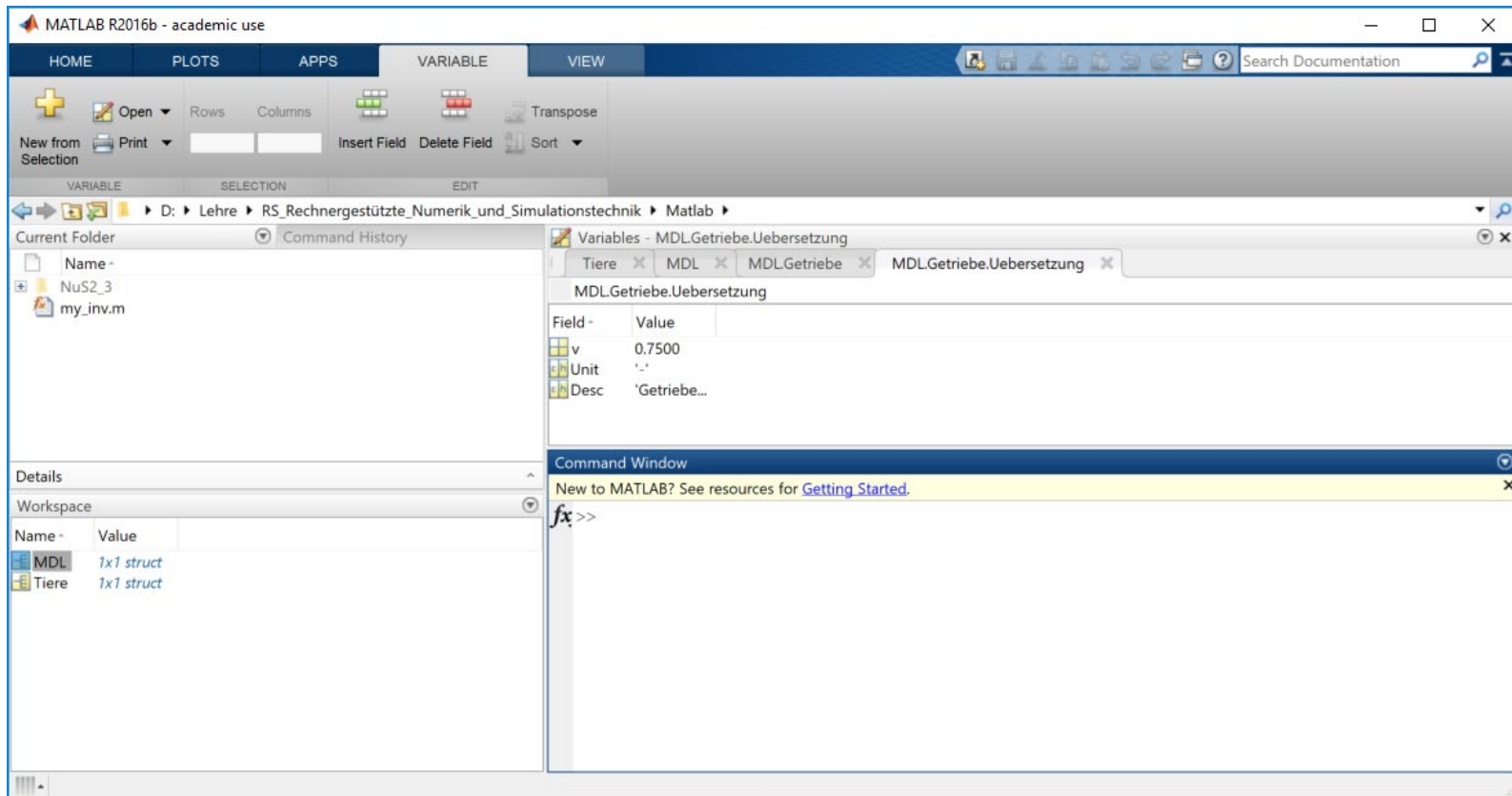
The screenshot shows a MATLAB command window with the following content:

```
>>  
>>  
>>  
>> Tiere.  
>> Tiere.Kaninchen
```

A tab completion menu is displayed over the command line, showing two options: 'Kaninchen' (highlighted in grey) and 'Tiger' (highlighted in yellow).

3. Datentypen – 3.4 Besonderheiten

Der Variable-Browser und der Array-Editor ermöglichen das Umbenennen von Variablen und das Ändern von Werten.



Bitte selber ausprobieren!!!

3. Datentypen – 3.4 Besonderheiten

■ Vordefinierte Werte:

ans	Enthält das letzte Ergebnis eines Befehls, falls keine Zuweisung zu einer anderen Variable erfolgt ist.
pi	Enthält die Zahl π .
eps	Enthält die Zahl $=2^{-52}$.
realmin, realmax	Kleinst- und größter reeller Wert, der in MATLAB verfügbar sind.
Inf	(infinite: unendlich) steht für $1/(+0)$. Beachten Sie den Unterschied zu $1/(-0)$.
NaN	(Not a Number) steht für $0/0$.
i, j	Enthalten beide die imaginäre Einheit $\sqrt{-1}$ zur Eingabe von komplexen Zahlen.

3. Datentypen – 3.4 Besonderheiten

Dünn besetzte Matrizen

- Effizientes Format für Matrizen mit vielen Elemente gleich 0 ist: Sparse-Format Nur Seicherung der von 0 verschiedenen Elemente und deren Indizes. Alle Matrixoperationen lassen sich auch auf das Sparse-Format anwenden.

```
>> A = [1, 0; 0, 2];
```

```
>> sparse(A)
```

```
ans =
```

```
(1,1)      1
```

```
(2,2)      2
```

```
>> S=sprand(10,10,.05)
```

```
S =
```

```
(9,1)      0.9572
```

```
(10,3)     0.4854
```

```
(2,6)      0.1576
```

```
(7,10)     0.9706
```

```
(10,10)    0.8003
```

3. Datentypen – 3.4 Besonderheiten

Spezielle Funktionen für Sparse-Matrizen:

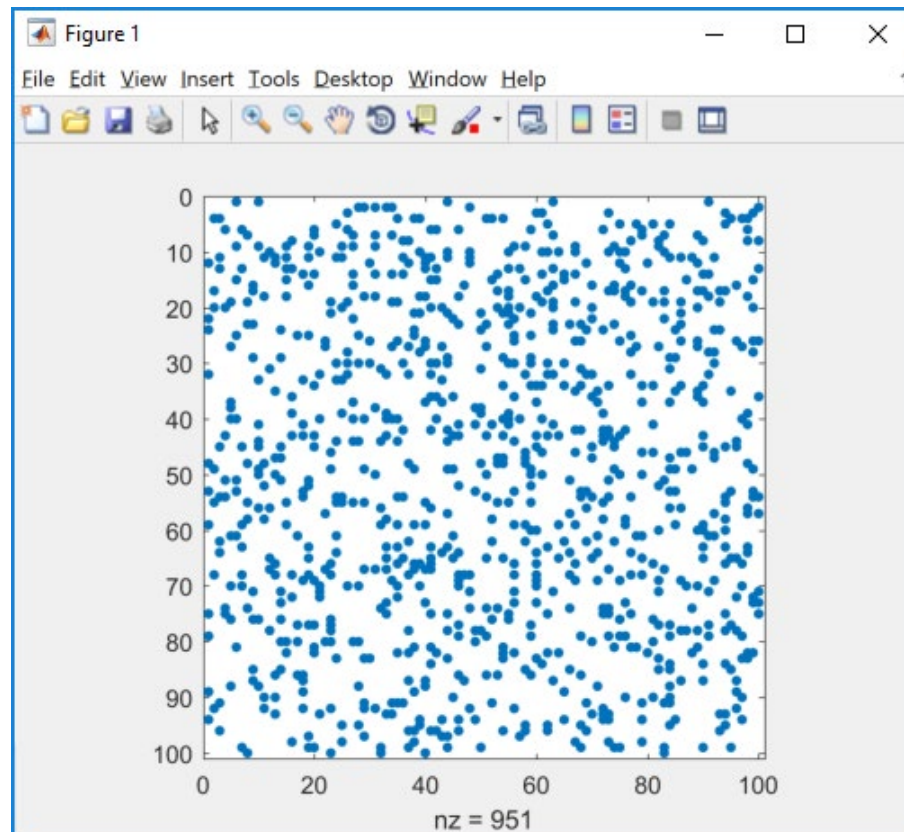
- `sparse, full` Konvertierung zwischen Sparse-Format und voll besetzten Matrizen
- `speye, spdiags` Einheits-/Diagonalmatrizen im Sparse-Format
- `sprand, sprandn` Sparse-Matrizen mit Pseudo-Zufallszahlen
- `nonzeros, nnz` Ausgabe der von 0 verschiedenen Elemente bzw. deren Anzahl
- `spy` Grafische Anzeige der Belegungsstruktur

Funktionen für Sparse-Matrizen:

- `eigs, svds` Bestimmung einiger Eigen- bzw. Singulärwerte
- `normest, condest` Norm- bzw. Konditionsschätzer
- `pcg, minres` Iterative Löser für dünnbesetzte Systeme

3. Datentypen – 3.4 Besonderheiten

```
A=sprand(100,100,.1);  
spy(A);
```



3. Datentypen – 3.5 Import und Export

Matlab-Standarddatenfile:

Endung: <filename>.mat

Speichern und Laden von Variablen in bzw. aus einer Datei:

```
>> save <filename>
```

```
>> load <filename>
```

für alle Workspace-Variablen oder für ausgewählte Variablen

```
>> save <filename> <variablenname1> <variablenname1>
```

```
>> load <filename> <variablenname1> <variablenname1>
```



3. Datentypen – 3.5 Import und Export

Beispiel:

```
>> a = 1; b = 2; c = 3;  
>> save data  
>> clear  
>> load data a  
>> who
```

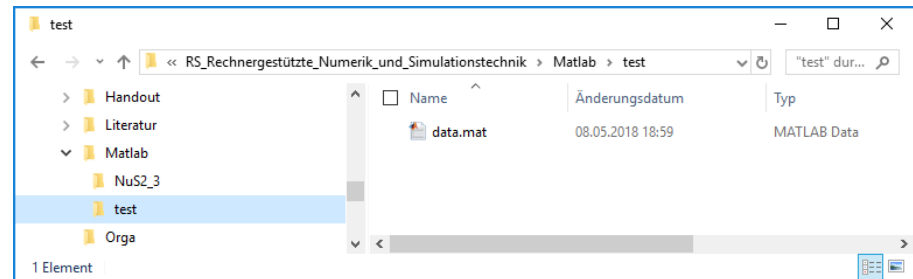
Your variables are:

a

```
>> load data  
>> who
```

Your variables are:

a b c



3. Datentypen – 3.5 Import und Export

Text-Files:

```
>> load data.txt
```

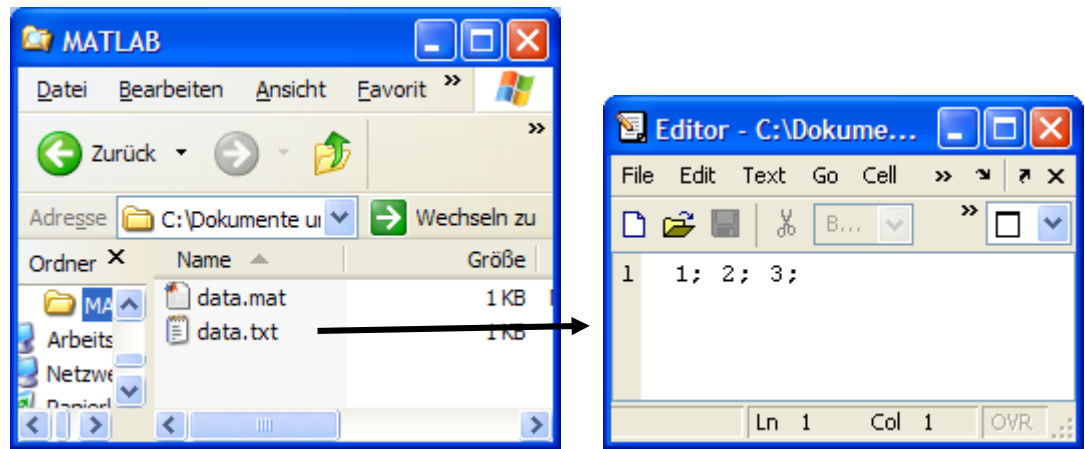
```
>> whos
```

Name	Size	Bytes	Class	Attributes
data	1x3	24	double	

```
>> data
```

```
data =
```

```
1      2      3
```



3. Datentypen – 3.5 Import und Export

Weitere Befehle zum Daten Import und Export:

uisave, uiload → Speichern/Laden mit GUI
diary → Speichern der Command History

Elementare Datei-Operation:

fopen → Öffnen einer Datei
fclose → Schließen einer Datei
fprintf → Schreiben von Text-Daten
fgets → Lesen von Text-Daten

3. Datentypen – 3.5 Import und Export

Beispiel 1:

```
>> filehandle = fopen('data.txt');  
>> line = fgets(filehandle)  
  
line =  
  
1; 2; 3;  
  
>> fclose(filehandle);
```

3. Datentypen – 3.5 Import und Export

Beispiel 2:

```
>> filehandle = fopen('data2.dat','w');  
>> fwrite(filehandle,100);  
>> fwrite(filehandle,120);  
>> fwrite(filehandle,130);  
>> fclose(filehandle);  
>> clear;
```

```
>> filehandle = fopen('data2.dat','r');  
>> input = fread(filehandle)
```

input =

```
100  
120  
130
```

3. Datentypen – 3.5 Import und Export

```
>> help fileformats
Readable file formats.
```

Data formats

MAT - MATLAB workspace
 CSV - Comma separated numbers
 DAT - Formatted text
 DLM - Delimited text
 TAB - Tab separated text

Command

load
 csvread
 importdata
 dlmread
 dlmread

Returns

Variables in file.
 Double array.
 Double array.
 Double array.
 Double array.

Spreadsheet formats

XLS - Excel worksheet
 WK1 - Lotus 123 worksheet

xlsread
 wk1read

Double array and cell array.
 Double array and cell array.

Scientific data formats

CDF - Common Data Format
 FITS - Flexible Image Transport System
 HDF - Hierarchical Data Format v.4
 H5 - Hierarchical Data Format v.5

cdfread Cell array of CDF records.
 fitsread Primary or extension table ..
 hdfread HDF 4 or HDF-EOS 2 data set.
 hdf5read HDF 5 or HDF-EOS 5 dataset.

...

3. Datentypen – 3.5 Import und Export

```
>> [NUMERIC,TXT,RAW] = xlsread('Messdaten.xls','Tabelle1');  
>> NUMERIC
```

```
NUMERIC =
```

0.0010	0.0998
0.0020	0.1987
0.0030	0.2955

```
...
```

```
>> TXT
```

```
TXT =
```

'time'	'voltage'
--------	-----------

```
>> RAW
```

```
RAW =
```

'time'	'voltage'
[1.0000e-003]	[0.0998]
[0.0020]	[0.1987]
[0.0030]	[0.2955]

```
...
```